

Complete Guide to Pandas.

Table of Contents(ToC):

1. The Basics
2. Creating DataFrame
3. Treating null values
4. Modify/Add new column(s).
5. Deleting columns
6. Renaming columns
- 7.i. Slicing DataFrame
- 7.ii. Slicing using iloc and loc
8. Adding a row
9. Dropping row(s)
10. Sorting
11. Joins
12. Groupby

Importing modules.

```
In [1]: import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
```

Importing data.

For this tutorial, we will use the standard Titanic Dataset.

```
In [2]: train=pd.read_csv('train.csv')
df=train.copy()
```

1. The Basics

```
In [3]: # See the first 5 rows
```

```
df.head()
```

Out[3]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0

In [4]: *# last 5 rows.*
df.tail()

Out[4]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0

In [5]: *# n_samples x n_features*

```
df.shape
```

```
Out[5]: (891, 12)
```

```
In [6]: #List of all the columns  
df.columns
```

```
Out[6]: Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age',  
             'SibSp',  
             'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],  
            dtype='object')
```

```
In [7]: # Rows index  
df.index
```

```
Out[7]: RangeIndex(start=0, stop=891, step=1)
```

```
In [8]: # Values with their counts in a particular column  
df['Pclass'].value_counts()
```

```
Out[8]: Pclass  
3      491  
1      216  
2      184  
Name: count, dtype: int64
```

```
In [9]: # General description of dataset.  
df.describe()
```

```
Out[9]:
```

	PassengerId	Survived	Pclass	Age	SibSp	
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.308642
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.815812
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000

2. Creating DataFrame

```
In [10]: # empty data frame  
df_empty=pd.DataFrame()  
df_empty.head()
```

```
Out[10]: —
```

With INDEX

```
In [11]: # From dict
student_dict={'Name':['A','B','C'],'Age':[24,18,17],'Roll':[1,2,3]}
df_student=pd.DataFrame(student_dict).reset_index(drop=False) # with index
df_student.head()
```

```
Out[11]:
```

	index	Name	Age	Roll
0	0	A	24	1
1	1	B	18	2
2	2	C	17	3

Without INDEX

```
In [12]: # From dict
student_dict={'Name':['A','B','C'],'Age':[24,18,17],'Roll':[1,2,3]}
df_student=pd.DataFrame(student_dict).reset_index(drop=True) # without index
df_student.head()
```

```
Out[12]:
```

	Name	Age	Roll
0	A	24	1
1	B	18	2
2	C	17	3

3. Treating null values

```
In [13]: df.head()
```

```
Out[13]:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	Age
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	ST 3
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	:

```
In [14]: # on whole df.
df.isnull().sum()
```

```
Out[14]: PassengerId      0
Survived      0
Pclass        0
Name          0
Sex           0
Age          177
SibSp         0
Parch         0
Ticket        0
Fare          0
Cabin        687
Embarked      2
dtype: int64
```

```
In [15]: # on a particular column
df['Age'].isnull().sum()
```

```
Out[15]: 177
```

Impute null values

```
In [16]: df['Age'] = df['Age'].fillna(df['Age'].mean())
```

```
In [17]: print(df['Age'].isnull().sum())
```

4. Modify/Add new column(s).

Adding another column or modify a column.

```
In [18]: df.head()
```

```
Out[18]:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	Age
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	ST 3
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	:

```
In [19]: df['gender']=df['Sex'].map({"male":'0',"female":'1'})
df.head()
```

Out[19]:	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	Age
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PassengerId
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	Survived
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	Pclass
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	Sex

```
In [20]: string = 'ahmed,mostafa'
string.split(",")
```

```
Out[20]: ['ahmed', 'mostafa']
```

```
In [ ]: def add(a,b):
        return a+b

add=lambda a,b: a+b
add(1,2)
```

```
In [21]: # Finding last name and first name from Name column.
df['last_name']=df['Name'].apply(lambda x: x.split(',')[0])
df['first_name']=df['Name'].apply(lambda x: ' '.join(x.split(',')[1:]
```

```
In [22]: df.head()
```

Out[22]:	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	Age
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	ST 3
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	:

```
In [23]: # Sets to 1 for men in 3rd class.
df['Third&Men']=df.apply(lambda row: int(row['Pclass']==3 and row['Sex']=='male'), axis=1)
df.head()
```

Out[23]:	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	Age
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	ST 3
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	:

Calling some custom function.

```
In [27]: def findAgeGroup(age):
        if age<18:
            return 1
        elif age>=18 and age<40:
            return 2
        elif age>=40 and age<60:
            return 3
        else:
            return 4
df['Age_group']=df['Age'].apply(lambda x: findAgeGroup(x))
# Calling a custom function.
```

```
In [28]: df.head()
```

Out[28]:	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	Age
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	ST 3
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	:

5. Deleting columns

In [29]: `df.head()`

Out[29]:	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	Age
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PassengerId
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	Survived
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	Pclass
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	Sex

```
In [31]: df = df.drop(['PassengerId'],axis=1)
#df=df.drop(['PassengerId'],axis=1,inplace=True)
df.head()
```

Out[31]:	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fa
0	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.25
1	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.28
2	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.92
3	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.10
4	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.05

6. Renaming columns

```
In [32]: # Lets try to rename some columns.
df=df.rename(columns={'Survived':'survived', 'Name':'Full Name', 'last_name':'last_name'})
df.head()
```

Out[32]:

	survived	Pclass	Full Name	Sex	Age	SibSp	Parch	Ticket	Fa
0	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.250
1	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.280
2	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.920
3	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.100
4	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.050

Similarly, we can do for whichever column we want.

7.i Slicing DataFrame

In [33]: `df.head()`

Out[33]:

	survived	Pclass	Full Name	Sex	Age	SibSp	Parch	Ticket	Fa
0	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.250
1	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.280
2	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.920
3	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.100
4	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.050

In [35]: df[df['survived'] == 1]

Out[35]:

	survived	Pclass	Full Name	Sex	Age	SibSp	Parch	Ticket	Fa
1	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.280
2	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.920
3	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.100
8	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.130
			Nasser, Mrs. James (Josephine May)	female	34.0	1	0	5153	9.360

9	1	2	Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30
...	...	...	...	...	...	...	...	...	...
875	1	3	Najib, Miss. Adele Kiamie "Jane"	female	15.0	0	0	2667	7.
879	1	1	Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)	female	56.0	0	1	11767	83
880	1	2	Shelley, Mrs. William (Imanita Parrish Hall)	female	25.0	0	1	230433	26.
887	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.
889	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.

342 rows × 16 columns

```
In [36]: # All rows with pclass==3
df_third_class=df[df['Pclass']==3]
df_third_class.head()
```

Out[36]:

	survived	Pclass	Full Name	Sex	Age	SibSp	Parch	Ticket
0	0	3	Braund, Mr. Owen Harris	male	22.000000	1	0	A/5 21171
2	1	3	Heikkinen, Miss. Laina	female	26.000000	0	0	STON/O2. 3101282
4	0	3	Allen, Mr. William Henry	male	35.000000	0	0	373450
5	0	3	Moran, Mr. James	male	29.699118	0	0	330877
7	0	3	Palsson, Master. Gosta Leonard	male	2.000000	3	1	349909

```
In [37]: # Females with age > 60
df_aged=df[(df['Age']>60) & (df['gender']=="1")]
df_aged.head()
```

Out[37]:

	survived	Pclass	Full Name	Sex	Age	SibSp	Parch	Ticket	Fa
275	1	1	Andrews, Miss. Kornelia Theodosia	female	63.0	1	0	13502	77.95
483	1	3	Turkula, Mrs. (Hedwig)	female	63.0	0	0	4134	9.58
829	1	1	Stone, Mrs. George Nelson (Martha Evelyn)	female	62.0	0	0	113572	80.00

Note that all these three ladies were saved as they were senior citizen and women so they may have been given priority.

```
In [42]: # Selecting some columns.
df1=df[['Age', 'Pclass', 'gender']]
df1.head()
```

Out[42]:

	Age	Pclass	gender
0	22.0	3	0
1	38.0	1	1
2	26.0	3	1
3	35.0	1	1
4	35.0	3	0

```
In [41]: x= 1,2  
         type(x)
```

Out[41]: tuple

```
In [43]: # Select numerical columns only  
#numerics = ['int16', 'int32', 'int64', 'float16', 'float32', 'float64']  
  
df_num = df.select_dtypes(include='number')  
df_num.head()
```

Out[43]:

	survived	Pclass	Age	SibSp	Parch	Fare	Third&Men	Age_group
0	0	3	22.0	1	0	7.2500	1	2
1	1	1	38.0	1	0	71.2833	0	2
2	1	3	26.0	0	0	7.9250	0	2
3	1	1	35.0	1	0	53.1000	0	2
4	0	3	35.0	0	0	8.0500	1	2

```
In [44]: # categorical columns  
df_cat=df.select_dtypes(include=['object'])  
df_cat.head()
```

Out[44]:

	Full Name	Sex	Ticket	Cabin	Embarked	gender	Surname	Name
0	Braund, Mr. Owen Harris	male	A/5 21171	NaN	S	0	Braund	Mr. Owen Harris
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	PC 17599	C85	C	1	Cumings	Mrs. John Bradley (Florence Briggs Thayer Cumings)
2	Heikkinen, Miss. Laina	female	STON/O2. 3101282	NaN	S	1	Heikkinen	Miss. Laina Heikkinen
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	113803	C123	S	1	Futrelle	Mrs. Jacques Heath (Lily May Peel) Futrelle
4	Allen, Mr. William Henry	male	373450	NaN	S	0	Allen	Mr. William Henry Allen

7.ii Slicing using iloc and loc

iloc

In [45]: `df.head()`

Out[45]:

	survived	Pclass	Full Name	Sex	Age	SibSp	Parch	Ticket	Fa
0	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.250
1	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.280
2	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.920
3	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.100
4	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.050

```
In [46]: # First 100 rows & all columns
df_sub1=df.iloc[0:100,:]
df_sub1
```

Out[46]:	survived	Pclass	Full Name	Sex	Age	SibSp	Parch	Ticl
0	0	3	Braund, Mr. Owen Harris	male	22.000000	1	0	A/5 21
1	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.000000	1	0	PC 175
2	1	3	Heikkinen, Miss. Laina	female	26.000000	0	0	STON/(31012
3	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.000000	1	0	1138
4	0	3	Allen, Mr. William Henry	male	35.000000	0	0	3734
...	...	...	...	...	...	...	...	...
95	0	3	Shorney, Mr. Charles Joseph	male	29.699118	0	0	3749
96	0	1	Goldschmidt, Mr. George B	male	71.000000	0	0	PC 177
97	1	1	Greenfield, Mr. William Bertram	male	23.000000	0	1	PC 177
98	1	2	Doling, Mrs. John T (Ada Julia Bone)	female	34.000000	0	1	2319
99	0	2	Kantor, Mr. Sinai	male	34.000000	1	0	2443

100 rows x 16 columns

```
In [47]: #First 250 rows with a subset of columns

#df_sub2=df.iloc[:250,['Age']]
# This will throw an error as iloc only consumes integres as indice.

df_sub2=df.iloc[:250,[1,8]]
#Returns first 250 rows and columns at those indices in df.columns.
df_sub2.head()
```

```
Out[47]:
```

	Pclass	Fare
0	3	7.2500
1	1	71.2833
2	3	7.9250
3	1	53.1000
4	3	8.0500

loc

```
In [48]: df.head()
```

```
Out[48]:
```

	survived	Pclass	Full Name	Sex	Age	SibSp	Parch	Ticket	Fa
0	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500
1	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...)	female	38.0	1	0	PC 17599	71.2833
2	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250
3	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000
4	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500

```
In [49]: # First 500 rows.
df_sub3=df.loc[:500,:]
df_sub3
```

Out[49]:

	survived	Pclass	Full Name	Sex	Age	SibSp	Parch	Ticket
0	0	3	Braund, Mr. Owen Harris	male	22.000000	1	0	A/5 2117
1	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.000000	1	0	PC 1759
2	1	3	Heikkinen, Miss. Laina	female	26.000000	0	0	STON/O: 310128
3	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.000000	1	0	11380
4	0	3	Allen, Mr. William Henry	male	35.000000	0	0	37345
...	...	...	...	...	...	...	...	...
496	1	1	Eustis, Miss. Elizabeth Mussey	female	54.000000	1	0	3694
497	0	3	Shellard, Mr. Frederick William	male	29.699118	0	0	C.A. 621
498	0	1	Allison, Mrs. Hudson J C (Bessie Waldo Daniels)	female	25.000000	1	2	11378
499	0	3	Svensson, Mr. Olof	male	24.000000	0	0	35003
500	0	3	Calic, Mr. Petar	male	17.000000	0	0	31508

501 rows × 16 columns

```
In [50]: # Gender and age of age >50
df_sub4=df.loc[(df['Age']>50),['gender','Age']]
df_sub4.head()
```

```
Out[50]:
```

	gender	Age
6	0	54.0
11	1	58.0
15	1	55.0
33	0	66.0
54	0	65.0

8. Adding a row

```
In [51]: df.head()
```

```
Out[51]:
```

	survived	Pclass	Full Name	Sex	Age	SibSp	Parch	Ticket	Fa
0	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500
1	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...)	female	38.0	1	0	PC 17599	71.2833
2	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250
3	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000
4	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500

```
In [ ]: len(df)
```

```
In [52]: df.tail()
```

Out[52]:

	survived	Pclass	Full Name	Sex	Age	SibSp	Parch	Ticket
886	0	2	Montvila, Rev. Juozas	male	27.000000	0	0	211536
887	1	1	Graham, Miss. Margaret Edith	female	19.000000	0	0	112053
888	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	29.699118	1	2	W./C. 6607
889	1	1	Behr, Mr. Karl Howell	male	26.000000	0	0	111369
890	0	3	Dooley, Mr. Patrick	male	32.000000	0	0	370376

In [55]: `len(df)`

Out[55]: 892

In [58]: `# Adding new row using loc
row = {'Survived':1, 'Pclass':np.nan, 'Full Name':'Peter', 'Gender':np
df.loc[len(df)]=row
df.tail()`

Out[58]:

	survived	Pclass	Full Name	Sex	Age	SibSp	Parch	Ticket
888	0.0	3.0	Johnston, Miss. Catherine Helen "Carrie"	female	29.699118	1.0	2.0	W./C. 6607
889	1.0	1.0	Behr, Mr. Karl Howell	male	26.000000	0.0	0.0	111369
890	0.0	3.0	Dooley, Mr. Patrick	male	32.000000	0.0	0.0	370376
891	NaN	NaN	Peter	NaN	NaN	NaN	NaN	NaN
892	NaN	NaN	Peter	NaN	NaN	NaN	NaN	NaN

In [56]: `li = [1,2,3]
li[3]=4`

IndexError

Traceback (most recent call

l last)

Cell In[56], line 2

1 li = [1,2,3]

----> 2 li[3]=4

IndexError: list assignment index out of range

9. Dropping row(s)

```
In [60]: df=df.drop(df.index[-1],axis=0) # Deletes last row  
df.tail()
```

Out[60]:

	survived	Pclass	Full Name	Sex	Age	SibSp	Parch	Ticket
886	0.0	2.0	Montvila, Rev. Juozas	male	27.000000	0.0	0.0	211536
887	1.0	1.0	Graham, Miss. Margaret Edith	female	19.000000	0.0	0.0	112053
888	0.0	3.0	Johnston, Miss. Catherine Helen "Carrie"	female	29.699118	1.0	2.0	W./C. 6607
889	1.0	1.0	Behr, Mr. Karl Howell	male	26.000000	0.0	0.0	111369
890	0.0	3.0	Dooley, Mr. Patrick	male	32.000000	0.0	0.0	370376

10. Sorting

```
In [61]: df.head()
```

Out[61]:

	survived	Pclass	Full Name	Sex	Age	SibSp	Parch	Ticket	Fa
0	0.0	3.0	Braund, Mr. Owen Harris	male	22.0	1.0	0.0	A/5 21171	7.250
1	1.0	1.0	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1.0	0.0	PC 17599	71.280
2	1.0	3.0	Heikkinen, Miss. Laina	female	26.0	0.0	0.0	STON/O2. 3101282	7.920
3	1.0	1.0	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1.0	0.0	113803	53.100
4	0.0	3.0	Allen, Mr. William Henry	male	35.0	0.0	0.0	373450	8.050

In [63]: `# sorting by age say in decreasing order.`
`df=df.sort_values(by=['Age','Pclass'],ascending=False) # can specify`
`df.head()`

Out[63]:

	survived	Pclass	Full Name	Sex	Age	SibSp	Parch	Ticket	Fa
630	1.0	1.0	Barkworth, Mr. Algernon Henry Wilson	male	80.0	0.0	0.0	27042	30.0
851	0.0	3.0	Svensson, Mr. Johan	male	74.0	0.0	0.0	347060	7.7
96	0.0	1.0	Goldschmidt, Mr. George B	male	71.0	0.0	0.0	PC 17754	34.6
493	0.0	1.0	Artagaveytia, Mr. Ramon	male	71.0	0.0	0.0	PC 17609	49.5
116	0.0	3.0	Connors, Mr. Patrick	male	70.5	0.0	0.0	370369	7.7

11. Joins

Let's create two separate data frames and try all 5 (Cross,Inner, Left,

Right, and Outer Joins).

```
In [64]: sno=[i+1 for i in range(100)]
marks=np.random.randint(100,size=100)
print(len(marks))
marks_df=pd.DataFrame({'Sno':sno,'Marks':marks})
marks_df.head()
```

100

```
Out[64]:
```

	Sno	Marks
0	1	17
1	2	81
2	3	85
3	4	37
4	5	92

```
In [65]: sno=[i+1 for i in range(100)]
age=np.random.randint(100,size=100)
print(len(marks))
age_df=pd.DataFrame({'Sno':sno,'Age':age})
age_df.head()
```

100

```
Out[65]:
```

	Sno	Age
0	1	71
1	2	27
2	3	54
3	4	29
4	5	57

Cross join

```
In [66]: cross_join=pd.merge(marks_df,age_df,how='cross')
print(cross_join.shape)
cross_join.head()
```

(10000, 4)

```
Out[66]:
```

	Sno_x	Marks	Sno_y	Age
0	1	17	1	71
1	1	17	2	27
2	1	17	3	54
3	1	17	4	29
4	1	17	5	57

Inner Join

```
In [67]: inner_join=pd.merge(age_df,marks_df,how='inner',on='Sno') # 'how' de
print(inner_join.shape)
inner_join.head()

(100, 3)
```

```
Out[67]:
```

	Sno	Age	Marks
0	1	71	17
1	2	27	81
2	3	54	85
3	4	29	37
4	5	57	92

Left and Right Outer Join(s)

Lets add some more rows to either frame so that left and right outer join makes sense

```
In [68]: age_df.loc[len(age_df)]={'Sno':101,'Age':23}
age_df.loc[len(age_df)]={'Sno':102,'Age':27}
age_df.loc[len(age_df)]={'Sno':104,'Age':29}
age_df.loc[len(age_df)]={'Sno':103,'Age':32}
age_df.loc[len(age_df)]={'Sno':105,'Age':53}
```

```
In [69]: #left join
left_join=pd.merge(age_df,marks_df,how='left',on='Sno')
print(left_join.shape)
left_join

(105, 3)
```

Out[69]:

	Sno	Age	Marks
0	1	71	17.0
1	2	27	81.0
2	3	54	85.0
3	4	29	37.0
4	5	57	92.0
...	...	...	...
100	101	23	NaN
101	102	27	NaN
102	104	29	NaN
103	103	32	NaN
104	105	53	NaN

105 rows × 3 columns

Marks=NaN confirms the left join.

```
In [70]: #right join
right_join=pd.merge(marks_df,age_df,how='right',on='Sno')
print(right_join.shape)
right_join.tail()
```

(105, 3)

Out[70]:

	Sno	Marks	Age
100	101	NaN	23
101	102	NaN	27
102	104	NaN	29
103	103	NaN	32
104	105	NaN	53

Marks=NaN confirms the right join in this case as the order of input frames is also reversed.

12. Groupby

Let's get back to our titanic dataset.

```
In [71]: df.head()
```

Out[71]:	survived	Pclass	Full Name	Sex	Age	SibSp	Parch	Ticket	
630	1.0	1.0	Barkworth, Mr. Algernon Henry Wilson	male	80.0	0.0	0.0	27042	30.0
851	0.0	3.0	Svensson, Mr. Johan	male	74.0	0.0	0.0	347060	7.7
96	0.0	1.0	Goldschmidt, Mr. George B	male	71.0	0.0	0.0	PC 17754	34.6
493	0.0	1.0	Artagaveytia, Mr. Ramon	male	71.0	0.0	0.0	PC 17609	49.5
116	0.0	3.0	Connors, Mr. Patrick	male	70.5	0.0	0.0	370369	7.7

Before, we move onto Groupby in pandas, lets recollect groupby from SQL. A SQL query, may contain hese parts (some might be optional like having etc...):

1. Table name (**from**)
2. Some condition to filter rows (**where clause**)
3. Attribute to groupby on (**groupby attributes**)
4. Filter rows after groupby (**having clause**)
5. Display attributes in the final result. (**select**)

Let's now see how to mimic this in Pandas ;)

```
In [72]: df.head()
```

Out[72]:	survived	Pclass	Full Name	Sex	Age	SibSp	Parch	Ticket	
630	1.0	1.0	Barkworth, Mr. Algernon Henry Wilson	male	80.0	0.0	0.0	27042	30.0
851	0.0	3.0	Svensson, Mr. Johan	male	74.0	0.0	0.0	347060	7.7
96	0.0	1.0	Goldschmidt, Mr. George B	male	71.0	0.0	0.0	PC 17754	34.6
493	0.0	1.0	Artagaveytia, Mr. Ramon	male	71.0	0.0	0.0	PC 17609	49.5
116	0.0	3.0	Connors, Mr. Patrick	male	70.5	0.0	0.0	370369	7.7

```
In [73]: groups=df.groupby(['Pclass'])
# uncomment this:
```

```
# groups.groups
groups.get_group(3) # Give some another no say 2 or 3 for its Pclas.
```

```
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_21184\3592587135.py:4:
FutureWarning: When grouping with a length-1 list-like, you will need
to pass a length-1 tuple to get_group in a future version of pandas.
Pass `(name,)` instead of `name` to silence this warning.
  groups.get_group(3) # Give some another no say 2 or 3 for its Pclass.
```

Out[73]:

	survived	Pclass	Full Name	Sex	Age	SibSp	Parch	Ticket	
851	0.0	3.0	Svensson, Mr. Johan	male	74.00	0.0	0.0	347060	7.0
116	0.0	3.0	Connors, Mr. Patrick	male	70.50	0.0	0.0	370369	7.0
280	0.0	3.0	Duane, Mr. Frank	male	65.00	0.0	0.0	336439	7.0
483	1.0	3.0	Turkula, Mrs. (Hedwig)	female	63.00	0.0	0.0	4134	9.0
326	0.0	3.0	Nysveen, Mr. Johan Hansen	male	61.00	0.0	0.0	345364	6.0
...	...	...	...	...	...	...	...	...	...
386	0.0	3.0	Goodwin, Master. Sidney Leonard	male	1.00	5.0	2.0	CA 2144	46.0
381	1.0	3.0	Nakid, Miss. Maria ("Mary")	female	1.00	0.0	2.0	2653	15.0
469	1.0	3.0	Baclini, Miss. Helene Barbara	female	0.75	2.0	1.0	2666	19.0
644	1.0	3.0	Baclini, Miss. Eugenie	female	0.75	2.0	1.0	2666	19.0
803	1.0	3.0	Thomas, Master. Assad Alexander	male	0.42	0.0	1.0	2625	8.0

491 rows x 16 columns

Average Age per Pclass

```
In [74]: df_grp1=df.groupby(['Pclass'])
#df_grp1.head()
```

```
In [76]: df.groupby(['Pclass'])['Age'].mean()
```

```
Out[76]: Pclass
1.0      37.048118
2.0      29.866958
3.0      26.403259
Name: Age, dtype: float64
```

```
In [ ]:
```

```
In [75]: #Average Age per Pclass
df_grp1['Age'].mean()
```

```
Out[75]: Pclass
1.0      37.048118
2.0      29.866958
3.0      26.403259
Name: Age, dtype: float64
```

```
In [77]: # Min/Max age per Pclass
df_grp1['Age'].min() #max()
```

```
Out[77]: Pclass
1.0      0.92
2.0      0.67
3.0      0.42
Name: Age, dtype: float64
```

```
In [78]: # Count
df_grp1['Age'].count()
```

```
Out[78]: Pclass
1.0      216
2.0      184
3.0      491
Name: Age, dtype: int64
```

Using agg() function.

We can use the aggregate function to aggregate results based on some column or our custom condition

```
In [79]: df=train.copy()
df.head()
```

Out[79]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	Age
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	Pclass
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	ST
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	

In [80]: *# Average per Pclass using 'agg' this time*
df_grp2=df.groupby(['Pclass']).agg({'Age':'mean', 'Fare':'median'})
df_grp2.head()

Out[80]:

	Age	Fare
Pclass		
1	38.233441	60.2875
2	29.877630	14.2500
3	25.140620	8.0500

In [81]: *# Names of all the pasengers in that class.*
df_grp4=df.groupby(['Pclass']).agg({'Name':lambda x: ', '.join(x)})
df_grp4.head()

Out[81]:

	Name
Pclass	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...
2	Nasser, Mrs. Nicholas (Adele Achem), Hewlett, ...
3	Braund, Mr. Owen Harris, Heikkinen, Miss. Lain...

TASK:

- 1 Convert the dictionary into a pandas DataFrame.
- 2 Display the first 3 rows of the DataFrame.
- 3 Find the average marks of all students.
- 4 Find the maximum marks scored.
- 5 Display the names of students who belong to the CS department.
- 6 Add a new column called Passed:

True if Marks $\geq$ 80

False otherwise
- 7 Count how many students are in each department.

```
In [ ]: data = {  
    "Student_ID": [1, 2, 3, 4, 5],  
    "Name": ["Ali", "Sara", "Omar", "Mona", "Youssef"],  
    "Age": [20, 22, 21, 23, 20],  
    "Marks": [85, 90, 78, 92, 88],  
    "Department": ["CS", "IT", "CS", "IT", "CS"]  
}
```

Solution:

```
In [ ]: import pandas as pd  
  
data = {  
    "Student_ID": [1, 2, 3, 4, 5],  
    "Name": ["Ali", "Sara", "Omar", "Mona", "Youssef"],  
    "Age": [20, 22, 21, 23, 20],  
    "Marks": [85, 90, 78, 92, 88],  
    "Department": ["CS", "IT", "CS", "IT", "CS"]  
}  
  
df = pd.DataFrame(data)  
df.head(3)
```

```
In [ ]: average_marks = df['Marks'].mean()  
average_marks
```

```
In [ ]: max_marks = df['Marks'].max()  
max_marks
```

```
In [ ]: cs_students = df[df['Department'] == 'CS']['Name']  
cs_students
```

```
In [ ]: df['Passed'] = df['Marks'] >= 80
```

```
In [ ]: dept_count = df['Department'].value_counts()  
dept_count
```

```
In [ ]: df
```